LTMB13 - IDMC - CAI Module: CDI - CAI Integration: D8 Session Notes - 26.10.2026

Informatica IDMC – CAI and CDI Integration

Complete Guide with Architecture, Industry Use Cases, Practical Scenarios and Interview Questions

Technology: Informatica Intelligent Data Management Cloud (IDMC)

Topic: Cloud Application Integration (CAI) with Cloud Data Integration (CDI)

Audience: Junior Data Engineers / Informatica Developers

Level: Beginner to Intermediate

Recommended Duration: 1 Day (8 Hours)

Training Method: Business Problem → Concept → Architecture → Process Flow → Practical Implementation → Validation → Interview Preparation

---

PART 1: What Is CAI–CDI Integration?

1.1 Understanding CAI and CDI Individually

Before explaining how CAI and CDI work together, we must understand their individual responsibilities.

Cloud Data Integration (CDI)

CDI is responsible for moving, transforming and loading data between systems.

For example, an organization stores customer information in an Oracle database and wants to load that information into Snowflake.

CDI can perform the following operations:

1. Read customer records from Oracle.

2. Filter inactive customers.
3. Standardize customer names.
4. Remove duplicate records.
5. Load the processed records into Snowflake.

A typical CDI mapping looks like this:

```
ORACLE CUSTOMER TABLE
          |
          v
    SOURCE TRANSFORMATION
          |
          v
    FILTER TRANSFORMATION
          |
          v
   EXPRESSION TRANSFORMATION
          |
          v
    TARGET TRANSFORMATION
          |
          v
   SNOWFLAKE CUSTOMER TABLE
```

Key point: CDI performs data integration activities.

Cloud Application Integration (CAI)

CAI is responsible for coordinating applications, invoking services and automating business processes.

For example, a banking application receives a request to process customer information.

CAI can:

1. Receive the request.
2. Validate the customer information.
3. Invoke another application.
4. Trigger a data integration task.
5. Check the execution result.
6. Return an appropriate response.

A typical CAI process looks like this:

```
APPLICATION REQUEST
         |
         v
     CAI PROCESS
         |
         v
   VALIDATE REQUEST
         |
         v
    INVOKE SERVICE
         |
         v
    CHECK RESPONSE
         |
         v
   RETURN RESULT
```

What Happens When We Integrate CAI and CDI?

CAI and CDI work together to automate an end-to-end business workflow.

CAI controls when and why a data integration activity should run. CDI performs the actual data movement and transformation.

Consider this example:

A retail company receives a new sales file.

The company wants to validate the request, execute a sales data pipeline and notify the requesting application.

Instead of manually starting the CDI task, CAI can coordinate the operation.

```
            BUSINESS APPLICATION
                     |
                     v
               REST API REQUEST
                     |
                     v
         CLOUD APPLICATION INTEGRATION
                     |
                     v
              VALIDATE REQUEST
                     |
                     v
               START CDI TASK
                     |
                     v
            CLOUD DATA INTEGRATION
                     |
                     v
              READ SALES DATA
```

```
                |
                v
        TRANSFORM SALES
                |
                v
        LOAD INTO TARGET
                |
                v
         TASK EXECUTION
                |
                v
      CAI STATUS HANDLING
                |
                v
         BUSINESS RESPONSE
```

Important distinction: Starting a CDI task does not mean that the task has finished successfully. CAI must explicitly check the CDI run status when the business requires confirmation of completion.

PART 2: Why Do Organizations Integrate CAI with CDI?

2.1 Business Problem: Manual Data Pipeline Execution

Consider a company that receives customer transaction files from different branches.

Every day, an employee must:

1. Confirm that the file is available.
2. Open Informatica IDMC.
3. Start the CDI mapping task.
4. Wait for the task to complete.
5. Check whether the task succeeded.
6. Inform the business team.

This process is repetitive.

It also creates operational risks.

For example:

- The employee may forget to start the task.

- The wrong task may be executed.
- The task may fail without immediate notification.
- The business application may not know when processing is complete.

Solution: CAI–CDI Integration

CAI can automate the coordination.

```
FILE ARRIVAL EVENT / API REQUEST
               |
               v
          CAI PROCESS
               |
               v
        VALIDATE REQUEST
               |
               v
        START CDI TASK
               |
               v
       CHECK EXECUTION STATUS
               |
        +------+------+
        |             |
        v             v
     SUCCESS        FAILURE
        |             |
        v             v
   NOTIFY USER     HANDLE ERROR
```

This approach is useful when the pipeline must be triggered by a business event or application request.

For simple time-based scheduling, CDI's native scheduling capabilities may be sufficient. CAI is especially useful when additional application interactions, decisions or orchestration are required.

PART 3: CAI–CDI Integration Architecture

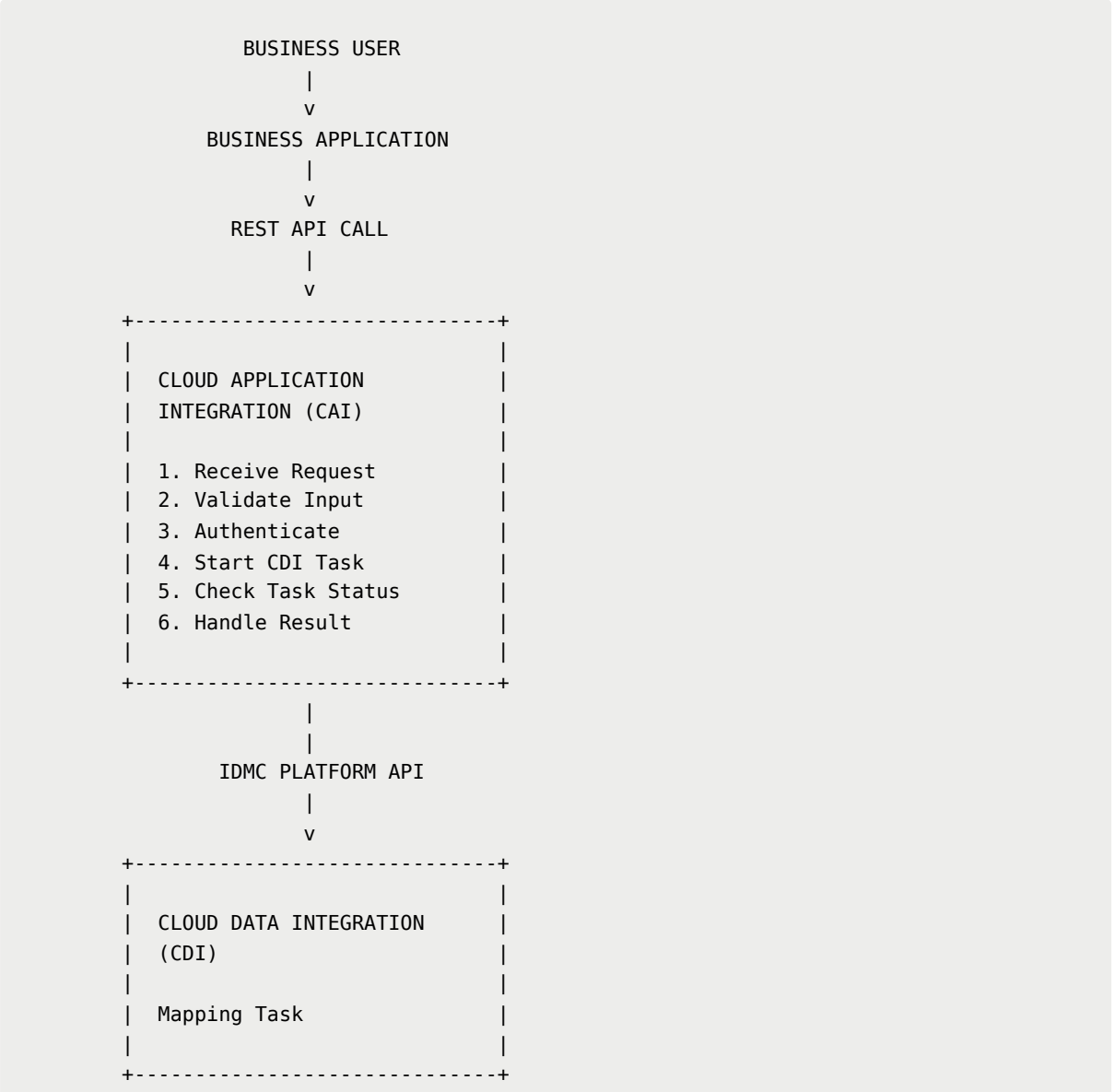
3.1 Understanding the Architecture

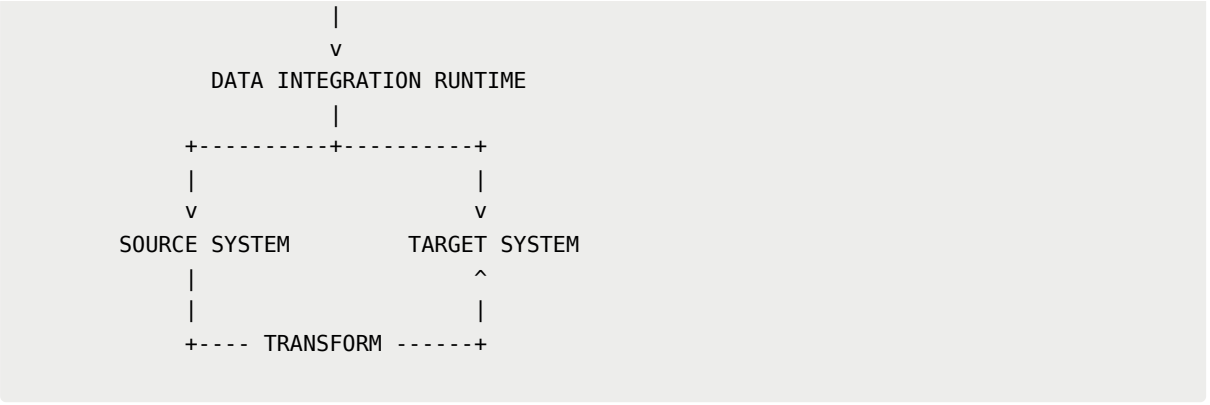
A typical CAI–CDI integration contains the following components:

Component	Responsibility
Business Application	Initiates a request

Component	Responsibility
CAI Process	Coordinates the workflow
REST API	Enables communication between services
IDMC Platform API	Provides supported operations for managing and executing IDMC assets
CDI Mapping Task	Executes data integration logic
Secure Agent / CDI Runtime	Runs the data integration workload
Source System	Supplies the data
Target System	Receives the processed data
Monitoring	Provides execution status and troubleshooting information

End-to-End Architecture





Explanation in Layman Terms:

Think of CAI as a **restaurant manager** and CDI as the **chef**.

The manager receives the customer's order and coordinates the work.

The chef prepares the food.

The manager then checks whether the order is ready and informs the customer.

Similarly:

- CAI receives and manages the business request.
- CDI performs the data processing.
- CAI checks the result and coordinates the next action.

This is a simple way to remember their responsibilities.

PART 4: How Does CAI Trigger a CDI Mapping Task?

This is one of the most important concepts in CAI–CDI integration.

4.1 Understanding the Execution Sequence

CAI can interact with CDI through supported Informatica services and APIs.

A common implementation uses the **IDMC Platform REST API**.

The process typically follows these stages:

1. Authenticate with the Informatica platform.
1. Obtain the required session or access token.
1. Identify the CDI task.

2. Submit a task execution request.
3. Obtain the execution identifier.
4. Check the execution status.
5. Handle success or failure.

Architecture

```
CAI PROCESS
     |
     v
LOGIN TO IDMC
     |
     v
GET AUTHENTICATION TOKEN
     |
     v
IDENTIFY CDI TASK
     |
     v
CALL CDI TASK EXECUTION API
     |
     v
RECEIVE EXECUTION IDENTIFIER
     |
     v
CHECK TASK STATUS
     |
     +------------------+
     |                  |
     v                  v
  SUCCESS            FAILURE
     |                  |
     v                  v
RETURN SUCCESS     HANDLE FAILURE
```

4.2 Understanding Authentication

Before CAI can invoke protected IDMC APIs, it must authenticate.

The exact authentication mechanism depends on the API family and organizational configuration.

For example, some Informatica Platform REST API workflows use a login operation that returns session information required by subsequent API calls.

Other APIs may use different authentication mechanisms.

Illustrative Login Request

```
{
  "username": "training_user",
  "password": "********"
}
```

Illustrative Login Response

```
{
  "sessionId": "EXAMPLE_SESSION_TOKEN",
  "serverUrl": "https://example.informaticacloud.com"
}
```

Training note: These are conceptual examples, not guaranteed request and response structures for every Informatica API version.

For an actual lab, use the authentication endpoint, payload, headers and response fields documented for the specific IDMC API version available in the training organization.

Credentials should be stored securely in connection configuration or an approved secret-management mechanism, rather than hardcoded in a process.

PART 5: Industry Scenario 1 – Retail Sales Processing

5.1 Business Requirement

A retail company has 50 branches.

Every branch uploads daily sales data.

The company wants to consolidate sales information into Snowflake.

The business requires:

1. Receive a request to process sales.
2. Validate the branch identifier.
3. Trigger the sales data integration pipeline.
4. Check whether processing succeeds.

5. Return the execution result.

Source Data: `SALES_RAW`

SALE_ID	BRANCH_ID	PRODUCT	QUANTITY	UNIT_PRICE
S1001	B101	Laptop	2	45000
S1002	B101	Mouse	5	500
S1003	B102	Keyboard	3	1200
S1004	B102	Monitor	2	15000
S1005	B103	Printer	1	12000

CDI Responsibility

The CDI mapping performs the data processing.

```
SOURCE: SALES_RAW
          |
          v
EXPRESSION TRANSFORMATION
          |
          | SALES_AMOUNT =
          | QUANTITY * UNIT_PRICE
          |
          v
TARGET: SALES_PROCESSED
```

Expected Target Data

SALE_ID	BRANCH_ID	PRODUCT	QUANTITY	SALES_AMOUNT
S1001	B101	Laptop	2	90000
S1002	B101	Mouse	5	2500
S1003	B102	Keyboard	3	3600
S1004	B102	Monitor	2	30000
S1005	B103	Printer	1	12000

CAI Responsibility

The CAI process coordinates the execution.

```
START
  |
```

```
  v
RECEIVE BRANCH REQUEST
  |
  v
VALIDATE BRANCH ID
  |
  v
START CDI SALES TASK
  |
  v
CHECK EXECUTION STATUS
  |
  +---- SUCCESS ----> RETURN SUCCESS
  |
  +---- FAILURE ----> RETURN ERROR
```

Important Implementation Consideration

If the API receives `BRANCH_ID = B101`, the CDI task must have a mechanism to process that specific branch.

For example, the mapping could use a parameter to filter sales records by branch.

However, **passing a value into a CAI process does not automatically pass it into a CDI mapping task**.

The implementation must explicitly configure and support the parameter transfer.

Interview Question

Q1. How can CAI and CDI be used together in a retail organization?

Answer:

CAI can receive a sales-processing request from a business application and trigger a CDI mapping task.

CDI performs the required data extraction, transformation and loading.

CAI can then check the task execution status and return the result to the requesting application.

This helps automate data pipelines that are initiated by business events.

PART 6: Industry Scenario 2 – Banking Transaction Processing

6.1 Business Requirement

A bank receives transaction data from multiple payment systems.

The bank wants to identify invalid transactions before loading valid records into a reporting database.

Examples of invalid transactions include:

- Negative amounts.
- Zero amounts.
- Missing amounts.
- Non-numeric amounts.
- Unsupported payment modes.

Example Source Data

TXN_ID	CUSTOMER	AMOUNT	PAYMENT_MODE
TXN1001	Ravi	5000	UPI
TXN1002	Priya	ABC	CARD
TXN1003	Arun	-2500	UPI
TXN1004	Meena	7500	NEFT
TXN1005	Karthik	NULL	CASH

CDI Responsibility

CDI performs the record-level data quality checks.

```
SOURCE: PAYMENTS_RAW
          |
          v
EXPRESSION TRANSFORMATION
          |
          v
VALIDATE AMOUNT
          |
          v
ROUTER TRANSFORMATION
          |
     +----+----+
     |         |
     v         v
 VALID       INVALID
     |         |
```

```
     v         v
VALID_TABLE  REJECT_TABLE
```

CAI Responsibility

CAI controls the overall business workflow.

```
PAYMENT BATCH REQUEST
          |
          v
       CAI PROCESS
          |
          v
   VALIDATE REQUEST
          |
          v
   START CDI MAPPING TASK
          |
          v
   CHECK EXECUTION STATUS
          |
          v
    RETURN BATCH RESULT
```

Important Business Distinction

A CDI task can finish successfully even when some source records are rejected according to business validation rules.

Therefore:

Technical success does not always mean that every transaction is valid.

For example:

- The CDI task processes 10,000 records.
- 9,700 records are valid.
- 300 records are rejected.
- The task completes successfully.

CAI should not assume that a successful task execution means zero business rejects.

If the business requires reject counts, the workflow must retrieve those counts from an appropriate source, such as an audit table or another reporting service.

Interview Question

Q2. What is the difference between a CDI task failure and a business validation failure?

Answer:

A CDI task failure is a technical execution problem, such as a connection error or mapping execution failure.

A business validation failure occurs when an individual record does not satisfy business rules.

For example, a transaction with a negative amount may be rejected by the mapping even though the CDI task completes successfully.

PART 7: Industry Scenario 3 – Healthcare Patient Data Integration

7.1 Business Requirement

A hospital receives patient registration information from multiple departments.

The organization wants to consolidate patient information into a central database.

The workflow must:

1. Receive a processing request.
2. Validate the request.
3. Trigger patient data integration.
4. Load standardized records.
5. Check the execution result.
6. Notify the appropriate application.

Architecture

```
HOSPITAL APPLICATION
         |
         v
    CAI REST API
         |
         v
    CAI PROCESS
         |
```

```
        v
  START CDI TASK
        |
        v
 PATIENT DATA MAPPING
        |
        v
STANDARDIZE PATIENT DATA
        |
        v
CENTRAL PATIENT DATABASE
        |
        v
  EXECUTION STATUS
```

CDI Transformation Examples

Source Field	Transformation	Target Field
patient_name	Trim unnecessary spaces	PATIENT_NAME
gender	Standardize supported values	GENDER
registration_date	Convert to required date format	REGISTRATION_DATE
hospital_id	Validate against reference data	HOSPITAL_ID

Interview Question

Q3. Why would a healthcare organization use CAI with CDI?

Answer:

CAI can coordinate patient-related integration workflows between applications.

CDI can perform the data extraction, standardization, validation and loading.

Together, they help automate the movement of information between authorized systems while supporting controlled workflow execution.

For healthcare data, security, privacy and access controls must also be implemented according to organizational and regulatory requirements.

PART 8: Industry Scenario 4 – E-Commerce Order Processing

8.1 Business Requirement

An online shopping company wants to update its analytics platform after receiving an order-processing event.

Business Flow

```
CUSTOMER PLACES ORDER
          |
          v
E-COMMERCE APPLICATION
          |
          v
       CAI PROCESS
          |
          v
  VALIDATE ORDER EVENT
          |
          v
  TRIGGER CDI PIPELINE
          |
          v
  LOAD ORDER ANALYTICS
          |
          v
  CHECK EXECUTION STATUS
          |
          v
  UPDATE PROCESS STATUS
```

Example

A customer places an order:

```
{
  "order_id": "ORD1001",
  "customer_id": "C101",
  "order_amount": 8500,
  "order_status": "CONFIRMED"
}
```

CAI receives the event.

It may trigger a CDI pipeline that loads order data into an analytics system.

Important Design Consideration

If hundreds of orders arrive every minute, starting a separate CDI batch task for every order may be inefficient.

A better design may be to collect order events and trigger processing in batches or use an appropriate event-driven data integration mechanism.

CAI–CDI integration should be designed according to transaction volume, latency requirements and processing cost.

Interview Question

Q4. Should CAI trigger a CDI mapping task for every customer order?

Answer:

Not necessarily.

For low-frequency business events, triggering a CDI task may be appropriate.

For high-volume transaction processing, it may be more efficient to batch events or use a suitable streaming or event-driven architecture.

The design depends on business latency requirements, processing volume and system capabilities.

PART 9: Practical Lab – Trigger a CDI Mapping Task from CAI

Lab Title

Automate Sales Data Processing Using CAI and CDI

Difficulty: Beginner to Intermediate

Suggested Duration: 90–120 Minutes

Objective: Create a CDI mapping task and invoke it from a CAI process using supported IDMC API operations.

This lab is designed as an implementation blueprint. Exact API URLs, request schemas and CAI configuration fields must be verified against the API version available in the training organization.

9.1 Prerequisites

Students require:

1. Access to an Informatica IDMC organization.
2. Cloud Data Integration access.

3. Cloud Application Integration access.
4. Permission to create CDI mappings and mapping tasks.
5. Permission to create and publish CAI processes.
6. An operational CDI runtime environment.
7. Access to source and target systems.
8. An account authorized to invoke the required IDMC APIs.
9. Postman or another API testing tool, where permitted.

Lab Architecture

```
POSTMAN / APPLICATION
          |
          v
     CAI PROCESS
          |
          v
   AUTHENTICATE TO IDMC
          |
          v
   START CDI MAPPING TASK
          |
          v
      CDI RUNTIME
          |
          v
     SOURCE SALES
          |
          v
    CALCULATE REVENUE
          |
          v
     TARGET SALES
          |
          v
    CHECK RUN STATUS
```

Task 1: Create the Source Table

For a database-based exercise, create a source table named:

`SALES_RAW`

Example SQL:

```
CREATE TABLE SALES_RAW
(
    SALE_ID VARCHAR(20),
    BRANCH_ID VARCHAR(10),
    PRODUCT_NAME VARCHAR(100),
    QUANTITY INTEGER,
    UNIT_PRICE DECIMAL(12,2)
);
```

Insert sample records:

```
INSERT INTO SALES_RAW
(
    SALE_ID,
    BRANCH_ID,
    PRODUCT_NAME,
    QUANTITY,
    UNIT_PRICE
)
VALUES
('S1001','B101','Laptop',2,45000),
('S1002','B101','Mouse',5,500),
('S1003','B102','Keyboard',3,1200),
('S1004','B102','Monitor',2,15000),
('S1005','B103','Printer',1,12000);
```

Task 2: Create the Target Table

```
CREATE TABLE SALES_PROCESSED
(
    SALE_ID VARCHAR(20),
    BRANCH_ID VARCHAR(10),
    PRODUCT_NAME VARCHAR(100),
    QUANTITY INTEGER,
    UNIT_PRICE DECIMAL(12,2),
    SALES_AMOUNT DECIMAL(14,2)
);
```

Task 3: Create the CDI Mapping

Open Cloud Data Integration.

Create a mapping named:

`M_SALES_PROCESSING`

Add the following transformations:

```
SOURCE
  |
  v
EXPRESSION
  |
  v
TARGET
```

Configure the source as `SALES_RAW`.

In the Expression transformation, calculate:

```
SALES_AMOUNT = QUANTITY * UNIT_PRICE
```

Configure the target as `SALES_PROCESSED`.

Validate and save the mapping.

Note: Use the expression syntax and data types supported by the configured CDI runtime and connector.

Task 4: Create the Mapping Task

Create a mapping task named:

`MT_SALES_PROCESSING`

Associate it with the mapping:

`M_SALES_PROCESSING`

Configure the runtime environment and the required connections.

Save the mapping task.

Initial Validation

Execute the mapping task manually.

Verify that five records are loaded into `SALES_PROCESSED`.

```sql
SELECT *
FROM SALES_PROCESSED
ORDER BY SALE_ID;
```

Expected result:

SALE_ID	BRANCH_ID	PRODUCT_NAME	SALES_AMOUNT
S1001	B101	Laptop	90000
S1002	B101	Mouse	2500
S1003	B102	Keyboard	3600
S1004	B102	Monitor	30000
S1005	B103	Printer	12000

Important: Before the automated run, configure an appropriate target-load strategy. Repeated inserts without cleanup or upsert logic may create duplicate target records.

PART 10: Create the CAI Process

Task 5: Create a CAI Process

Open Cloud Application Integration.

Create a process named:

`PRC_TRIGGER_SALES_CDI`

The process will coordinate the CDI task execution.

Suggested Process Inputs

Field	Data Type	Description
REQUEST_ID	String	Identifies the business request
REQUESTED_BY	String	Identifies the requesting application or user

Suggested Process Outputs

Field	Data Type	Description
REQUEST_ID	String	Original request identifier

Field	Data Type	Description
EXECUTION_ID	String	Identifier returned by the execution API
PROCESS_STATUS	String	Status returned by the CAI workflow
MESSAGE	String	Informational or error message

Task 6: Configure the Authentication Service

Create or configure the supported service connector and connection required to authenticate with the Informatica platform.

The connector should expose the relevant login operation.

The authentication response must provide the information required for subsequent authorized API calls.

Conceptual Flow

```
START
  |
  v
LOGIN SERVICE CALL
  |
  v
RECEIVE AUTHENTICATION RESULT
  |
  v
VALIDATE LOGIN RESPONSE
```

Security requirement: Do not expose authentication tokens or passwords in business API responses or unnecessary logs.

Task 7: Configure the CDI Task Execution Service

Create or configure a service operation for starting the CDI mapping task.

The operation must use the appropriate Informatica task execution API.

A commonly documented Informatica Cloud REST API pattern uses a task execution endpoint similar to:

```
POST /api/v2/job
```

For compatible API versions, a task execution request may use a structure similar to:

```
{
  "taskId": "TASK_IDENTIFIER",
  "taskType": "MT"
}
```

Here:

- `taskId` identifies the CDI mapping task.
- `taskType` identifies the type of task being executed.
- `MT` represents a mapping task in this API convention.

Important: The endpoint, authentication header, task identifier format and response schema are version-specific. Use the official API documentation for the actual training environment rather than treating this example as a universally valid request.

Task 8: Capture the Execution Response

When the task execution request is accepted, the API may return an execution identifier.

The CAI process should capture the relevant value.

For example:

```
EXECUTION_ID = returned execution identifier
```

The process should distinguish between:

- A request accepted for execution.
- A task currently running.
- A task completed successfully.
- A task completed with failure.

Important Concept

A successful API response to the task-start request does not necessarily mean that the CDI task has completed.

This distinction is frequently tested in interviews.

Task 9: Check CDI Execution Status

Use the appropriate IDMC job-monitoring or execution-status API supported by the environment.

A typical workflow is:

```
GET EXECUTION STATUS
          |
          v
      CHECK STATUS
          |
    +-----+-----+
    |           |
    v           v
RUNNING      COMPLETED
    |           |
    v           v
WAIT / RETRY  CHECK RESULT
                |
           +----+----+
           |         |
           v         v
        SUCCESS    FAILURE
```

The implementation should use a bounded polling strategy.

For example:

- Check periodically.
- Stop when a terminal status is reached.
- Stop when a configured timeout is exceeded.
- Handle API failures separately from CDI task failures.

Avoid unlimited polling loops.

Task 10: Return the Final Result

Example successful response:

```
{
  "request_id": "REQ1001",
  "execution_id": "EXAMPLE_RUN_ID",
  "process_status": "SUCCESS",
```

```
  "message": "Sales processing completed successfully"
}
```

Example failed response:

```
{
  "request_id": "REQ1002",
  "execution_id": "EXAMPLE_RUN_ID",
  "process_status": "FAILED",
  "message": "Sales processing did not complete successfull
y"
}
```

These are suggested business-facing CAI responses, not native Informatica API response schemas.

PART 11: Fault Handling in CAI–CDI Integration

Fault handling is essential because the workflow involves multiple services.

Failures can occur at different stages.

11.1 Common Failure Scenarios

Failure	Possible Cause	Recommended Handling
Authentication failure	Invalid credentials or permissions	Stop the workflow and record the failure
CDI task not found	Incorrect task identifier	Validate task configuration
Task execution rejected	Missing permission or invalid request	Capture API error
CDI task failure	Source, transformation or target issue	Inspect CDI execution logs
Monitoring API failure	Connectivity or service problem	Apply bounded retry where appropriate
Execution timeout	Task exceeds allowed waiting period	Return timeout or pending status

Failure	Possible Cause	Recommended Handling
Duplicate execution	Same business request submitted twice	Apply request deduplication or idempotency controls

11.2 Example Fault Handling Flow

```
START
  |
  v
AUTHENTICATE
  |
  +---- FAILURE ----> AUTHENTICATION ERROR
  |
  v
START CDI TASK
  |
  +---- FAILURE ----> EXECUTION REQUEST ERROR
  |
  v
CHECK TASK STATUS
  |
  +---- RUNNING ----> WAIT / CHECK AGAIN
  |
  +---- FAILED -----> CDI TASK ERROR
  |
  +---- TIMEOUT ----> TIMEOUT HANDLING
  |
  v
SUCCESS
```

Interview Question

Q5. How do you handle failures when CAI triggers a CDI mapping task?

Answer:

I would implement error handling for authentication, task execution and status monitoring.

If authentication fails, the process should stop and record the error.

If the CDI task starts but later fails, CAI should capture the execution status and report the failure.

I would also configure appropriate timeout handling and avoid uncontrolled retries that might create duplicate processing.

PART 12: Monitoring CAI and CDI Together

A key operational challenge is that CAI and CDI have different execution responsibilities.

Therefore, monitoring must cover both.

12.1 CAI Monitoring

CAI monitoring focuses on the application integration process.

Typical questions include:

- Did the CAI process start?
- Did authentication succeed?
- Was the CDI task execution request accepted?
- Did the status-checking logic execute correctly?
- Did the process encounter a fault?

12.2 CDI Monitoring

CDI monitoring focuses on the data integration task.

Typical questions include:

- Did the mapping task start?
- Did the source connection succeed?
- How many records were processed?
- Were records rejected?
- Did the target load succeed?
- Did the task complete successfully?

Combined Monitoring Example

Component	Example Status	Meaning
CAI Process	Running	Orchestration is in progress
CDI Task	Running	Data processing is in progress
CAI Process	Completed	CAI workflow finished
CDI Task	Failed	Data integration encountered a failure

A completed CAI process does not automatically establish that the CDI task succeeded.

The CAI process must explicitly interpret the CDI result.

Interview Question

Q6. Why do we need to monitor both CAI and CDI?

Answer:

CAI and CDI perform different operations.

CAI monitoring helps identify issues in process orchestration, API calls and business workflow execution.

CDI monitoring helps identify issues in data extraction, transformation and loading.

Monitoring both provides better visibility into the complete integration workflow.

PART 13: CAI–CDI Integration vs CDI Task Scheduling

We should understand when CAI is necessary.

Requirement	CDI Scheduling	CAI–CDI Integration
Run a task every day at 10 PM	Suitable	Usually unnecessary
Run a task after receiving an API request	Not sufficient by itself	Suitable
Validate business input before execution	Limited to task-related configuration	Suitable
Invoke multiple external services	Not the primary purpose	Suitable
Return a business API response	Not the primary purpose	Suitable
Coordinate complex application workflows	Limited	Suitable
Execute a standard batch ETL pipeline	Suitable	CAI optional
Trigger a pipeline based on application events	Requires additional integration mechanisms	Suitable

Explanation: Suppose the requirement is:

"Run the sales mapping every night at 11 PM."

Use CDI scheduling.

Now suppose the requirement is:

"When the finance application approves a monthly report, trigger the sales mapping, check the result and notify the reporting application."

CAI–CDI integration becomes useful because the workflow depends on a business event and requires additional coordination.

PART 14: Important Technical Concepts for Interviews

14.1 Synchronous vs Asynchronous Execution

Synchronous

The caller waits for the operation to finish and receive its result.

Example:

A customer validation API returns the validation result immediately.

Asynchronous

The operation starts, but processing continues independently of the original request.

Example:

CAI submits a CDI task execution request and receives an execution identifier.

The CDI task continues running.

CAI or another component can check its status later.

Interview Question

Q7. Is starting a CDI mapping task from CAI synchronous or asynchronous?

Answer:

A CDI task-start API call commonly initiates asynchronous task execution.

The API may return an execution identifier before the task finishes.

If the business requires the final outcome, CAI must implement additional status monitoring or another supported completion mechanism.

14.2 Idempotency

Idempotency means that repeating the same business request does not unintentionally produce additional effects.

Example

A finance application sends the same request twice because of a network retry.

Without duplicate protection:

```
REQUEST 1001
     |
     v
START CDI TASK
     |
     v
LOAD 100 RECORDS

REQUEST 1001 AGAIN
     |
     v
START CDI TASK AGAIN
     |
     v
LOAD SAME 100 RECORDS AGAIN
```

This may create duplicate data.

Possible controls include:

- Unique business request identifiers.
- Tracking previously processed requests.
- Target upsert logic.
- Duplicate detection.
- Appropriate execution controls.

Interview Question

Q8. How can duplicate CDI executions be prevented?

Answer:

I would use a unique request identifier and track whether the request has already been processed.

Depending on the business requirements, I would also implement target-side deduplication or upsert logic.

This helps prevent repeated API requests from producing duplicate data.

14.3 Parameterization

Parameterization allows the same integration logic to operate with different input values.

Example

A sales mapping processes one business date at a time.

Instead of creating separate mappings for each date, the mapping can use a parameter.

Conceptually:

```
PROCESS_DATE = 2026-10-25
```

The mapping uses this value to select the appropriate sales records.

Interview Question

Q9. Can CAI pass parameters to CDI?

Answer:

CAI can coordinate parameterized CDI execution when the selected integration mechanism and API support the required parameter configuration.

The CDI mapping and mapping task must be designed to accept those parameters.

The values must be passed using the supported API request format or another supported mechanism.

Passing a value into CAI alone does not automatically configure the CDI task.

PART 15: Interview Preparation – 25 Questions and Answers

Section A: Fundamentals

Q1. What is CAI–CDI integration?

CAI–CDI integration combines application process orchestration with data integration.

CAI coordinates business requests and task execution, while CDI performs the required data extraction, transformation and loading.

Q2. Why do organizations integrate CAI with CDI?

Organizations use this integration to automate data pipelines based on business events, application requests and workflow decisions.

It reduces manual coordination and enables application-driven data processing.

Q3. What is the main responsibility of CAI?

CAI manages application interactions, service calls, business decisions and process orchestration.

Q4. What is the main responsibility of CDI?

CDI performs data extraction, transformation and loading between supported source and target systems.

Q5. Can CAI execute a CDI mapping task?

Yes. CAI can invoke supported IDMC services or APIs to initiate CDI task execution, provided the required permissions and configurations are available.

Section B: Architecture and APIs

Q6. What is the role of REST APIs in CAI–CDI integration?

REST APIs allow CAI to communicate with supported Informatica platform services.

They can be used for operations such as authentication, task execution and status retrieval.

Q7. Why is authentication required?

Authentication verifies that the calling application or process is authorized to access the requested Informatica services.

Q8. What is a task identifier?

A task identifier is a value used by the relevant API to identify a specific integration task.

Q9. What is an execution identifier?

An execution identifier identifies a particular run of a task.

It can be used with supported monitoring operations to retrieve execution information.

Q10. What is the difference between a task identifier and an execution identifier?

A task identifier refers to the configured task.

An execution identifier refers to a particular execution of that task.

The same task can have multiple executions over time.

Section C: Execution and Monitoring

Q11. Does a successful task-start API response mean that the CDI task succeeded?

No.

It generally indicates that the execution request was accepted or initiated.

The final task status must be checked separately.

Q12. How can CAI determine whether a CDI task completed?

CAI can use supported execution-status APIs or other available completion mechanisms.

The process should interpret the returned status and distinguish running, successful and failed executions.

Q13. What happens if a CDI task fails?

The CAI workflow should capture the failure information and perform the configured fault-handling action.

This may include logging, returning an error or notifying another system.

Q14. Can CAI trigger multiple CDI tasks?

Yes.

A CAI process can coordinate multiple task execution requests.

The process design should account for dependencies, concurrency and error handling.

Q15. What is the difference between sequential and parallel task execution?

Sequential execution runs tasks according to a defined order.

Parallel execution allows independent tasks to run concurrently.

Sequential execution is appropriate when one task depends on another.

Parallel execution is appropriate when tasks are independent and the runtime supports the required concurrency.

Section D: Practical Scenarios

Q16. A company wants to execute a CDI mapping every night. Is CAI required?

No.

A standard CDI schedule may be sufficient.

CAI becomes useful when additional business workflow orchestration is required.

Q17. A company wants to trigger a CDI mapping after receiving an API request. Which service is suitable?

CAI can receive the API request, validate it and invoke the required CDI task through supported integration mechanisms.

Q18. A CDI mapping loads invalid transactions into a reject table but completes successfully. Should CAI report the entire batch as failed?

Not automatically.

The answer depends on the business rules.

CAI may need to retrieve reject counts or validation results to determine the correct business outcome.

Q19. How would you prevent duplicate processing when an API request is retried?

I would use request identifiers, execution tracking and appropriate target-side controls.

This helps ensure repeated requests do not create duplicate business results.

Q20. A CDI task takes 30 minutes. Should CAI keep the original HTTP request open for the entire duration?

Usually not.

A better design is often to return a request or execution identifier and allow the caller to check the status later.

This avoids long-running HTTP requests and timeout problems.

Section E: Fault Handling and Best Practices

Q21. What happens if authentication fails?

The process should stop the protected operation, capture the authentication error and return an appropriate failure response.

Q22. What happens if the CDI task cannot be found?

The process should report a task configuration or lookup error.

The developer should verify the task identifier, permissions and API request.

Q23. Why should API credentials not be hardcoded?

Hardcoded credentials create security and maintenance risks.

Credentials should be managed using approved secure configuration or secret-management mechanisms.

Q24. Why is monitoring important in CAI–CDI integration?

Monitoring helps identify whether failures occurred in the orchestration process, API communication or data integration task.

It supports troubleshooting and operational visibility.

Q25. What are the most important best practices for CAI–CDI integration?

The main best practices are:

- Use secure authentication.
- Validate business requests.
- Use supported APIs.
- Capture execution identifiers.
- Check final task status.
- Handle faults and timeouts.
- Prevent duplicate processing.
- Maintain useful execution logs.
- Separate development and production configurations.
- Avoid unnecessary task executions.

PART 16: Five Scenario-Based Interview Questions

These questions are particularly useful for interviews because they test understanding rather than memorized definitions.

Scenario 1: API Request Accepted but No Data Loaded

Question:

A CAI process calls the CDI task execution API.

The API returns a successful response.

However, no records appear in the target table.

What would you investigate?

Expected Answer:

I would first confirm whether the API response indicates only that the task was started.

I would then use the execution identifier to check the CDI task status.

If the task failed, I would inspect the CDI logs.

If it succeeded, I would verify the source records, mapping filters, target configuration and load results.

Scenario 2: Duplicate Records After API Retry

Question:

A business application sends the same request twice.

Both requests trigger the CDI task.

The target contains duplicate records.

How would you address this?

Expected Answer:

I would investigate whether duplicate request handling was implemented.

I would recommend using a unique request identifier, tracking processed requests and applying appropriate target-side deduplication or upsert logic.

Scenario 3: CDI Task Succeeds but Business Reports an Error

Question:

A CDI task successfully processes 10,000 payment transactions.

However, 200 records fail business validation.

Why might the business report an issue even though the task succeeded?

Expected Answer:

The task may have completed technically while rejecting invalid records according to business rules.

I would verify the reject count and determine whether the business acceptance criteria allow rejected records.

Scenario 4: CDI Task Takes Too Long

Question:

A CAI process triggers a CDI task that normally takes 40 minutes.

The calling application receives an HTTP timeout.

What design improvement would you suggest?

Expected Answer:

I would recommend an asynchronous workflow.

CAI can initiate the task and return an execution or tracking identifier.

The calling application can then retrieve the final status through a separate status-checking mechanism.

Scenario 5: Multiple Dependent CDI Tasks

Question:

A retail company has three CDI tasks:

1. Load customer data.
2. Load order data.
3. Generate customer sales summary.

The summary must run only after the first two tasks succeed.

How would you design the workflow?

Expected Answer:

I would coordinate the tasks based on their dependencies.

The customer and order loads may run independently if their requirements allow it.

After confirming that both completed successfully, the workflow can trigger the summary task.

If either prerequisite task fails, the summary should not run.

For purely data-pipeline dependencies, I would also evaluate CDI's native taskflow capabilities before selecting CAI.

PART 17: CAI vs CDI vs CDI Taskflow

This is an important additional topic for training.

Students should not assume that CAI is required for every multi-step data pipeline.

Feature	CAI	CDI Mapping	CDI Taskflow
Primary purpose	Application and service orchestration	Data transformation and movement	Data integration task orchestration
Receives application API requests	Supported	Not its primary purpose	Not its primary purpose
Performs row-level transformations	Not its primary purpose	Yes	No; invokes tasks that perform them
Coordinates mapping tasks	Possible through supported integration mechanisms	No	Yes
Invokes external services	Yes	Connector-dependent	Limited to supported taskflow capabilities
Handles application business logic	Yes	Limited	Primarily task orchestration logic
Typical use	Order-processing API workflow	Sales ETL mapping	Execute customer load, then sales load

When Should You Use Each?

Use CDI Mapping when:

The main requirement is to read, transform and load data.

Use CDI Taskflow when:

The main requirement is to coordinate data integration tasks, handle dependencies and control data pipeline execution.

Use CAI when:

The requirement involves application requests, external services, APIs, business decisions or broader application orchestration.

Example

Requirement A: Load sales data into Snowflake.

Recommended starting point: CDI mapping and mapping task.

Requirement B: Run customer and sales mappings in sequence.

Recommended starting point: CDI taskflow.

Requirement C: Receive a request from a finance application, validate the request, initiate a data integration workflow and communicate the result.

Recommended starting point: CAI coordinating the appropriate CDI execution mechanism.

PART 18: Recommended One-Day Training Schedule

Session	Topic	Duration	Training Activity
Session 1	CAI and CDI fundamentals	1 hour	Explain responsibilities with retail examples
Session 2	CAI–CDI architecture	1 hour	Architecture walkthrough
Session 3	IDMC APIs and authentication	1 hour	Demonstrate API request and response concepts
Session 4	CDI mapping and task creation	1 hour	Create sales processing mapping
Session 5	CAI process design	1 hour	Build the orchestration flow
Session 6	CDI task execution from CAI	1 hour	Configure supported service operations
Session 7	Monitoring and fault handling	1 hour	Test success and failure scenarios
Session 8	Industry scenarios and interview practice	1 hour	Scenario-based discussion and assessment

PART 19: Assessment – CAI–CDI Integration

Assessment Title: Sales Processing Automation Using CAI and CDI

Difficulty: Medium

Marks: 20

Suggested Duration: 25 Minutes

Prerequisite: A working CDI mapping task and the required authenticated API connection are provided to trainees. This keeps the assessment focused on CAI–CDI integration rather than environment setup.

Objective

Create a CAI process that invokes an existing CDI sales mapping task and captures the execution response.

Task 1: Create the CAI Process

Create a process named:

`PRC_SALES_AUTOMATION`

Configure an input field named `REQUEST_ID`.

Marks: 4

Task 2: Configure the CDI Execution Service

Use the provided service connector and connection to invoke the existing CDI mapping task.

Marks: 5

Task 3: Capture the Execution Identifier

Capture the execution identifier returned by the supported task execution API.

Marks: 4

Task 4: Configure Response Handling

Return a response indicating whether the task execution request was accepted or failed.

Do not report the CDI task as completed unless its final status has been verified.

Marks: 4

Task 5: Test and Validate

Execute the CAI process with a sample request.

Verify that the expected CDI task execution was initiated.

Marks: 3

Evaluation Rubric

Evaluation Area	Marks
CAI process and input configuration	4
CDI execution service configuration	5
Execution identifier capture	4
Response handling	4
Successful test and validation	3
Total	**20**

PART 20: Final Revision Notes for Understranding:

Remember These 10 Points

1. **CAI** is primarily used for application integration and business process orchestration.
2. **CDI** is primarily used for data extraction, transformation and loading.
3. CAI can coordinate CDI execution using supported APIs or integration mechanisms.
4. A CDI mapping defines the data transformation logic.
5. A CDI mapping task provides an executable configuration for the mapping.
6. An API task-start response does not necessarily mean that the task has completed.
7. An execution identifier helps track a specific task run.
8. CAI and CDI execution must be monitored separately when troubleshooting.
9. Fault handling, authentication and duplicate prevention are essential for reliable integration.
10. CDI taskflows should also be considered when the requirement is primarily data pipeline orchestration.

The Most Important Interview Answer

Question: Explain CAI–CDI integration with a real-time example.

Answer:

"Informatica CAI–CDI integration is used to automate data integration activities through application-driven workflows.

For example, in a retail organization, a business application can send a request to process daily sales data.

CAI receives the request, validates the information and invokes the appropriate CDI mapping task through a supported Informatica API.

CDI then reads the sales data, performs transformations and loads the processed information into the target system.

CAI can monitor the task execution status and communicate the result to the requesting application.

This integration is useful when data processing must be coordinated with business events, APIs or other application workflows."

Recommended Learning Sequence for → CAI–CDI integration in this exact order:

1. CAI vs CDI → 2. Business Problem → 3. CAI–CDI Architecture → 4. CDI Mapping Task → 5. IDMC REST APIs → 6. Authentication → 7. CAI Process Designer → 8. Invoke CDI Task → 9. Capture Execution ID → 10. Monitor Execution → 11. Fault Handling → 12. Industry Scenarios → 13. Interview Preparation.

This approach helps us to understand not only **how CAI and CDI are integrated**, but also **why organizations use the integration, when it is appropriate and how to troubleshoot it in real projects**.